JAVA FILE HANDLING

CHARACTER STREAMS

Complete Learning & Revision Guide

Lessons 6–15 | BufferedReader to Character Encoding, Practical Programs, Comparisons & Interview Preparation

Topics covered

- BufferedReader and BufferedWriter
- InputStreamReader and OutputStreamWriter
- PrintWriter and formatted output
- Unicode, UTF-8, UTF-16 and Charset
- Character stream hierarchy and class comparisons
- Practical Java programs, common mistakes and interview questions

How to use this guide

This guide covers the Character Streams lessons from BufferedReader (Lesson 6) through the final revision section (Lesson 15). Read definitions, trace the wrapper relationships, then type and run the programs yourself.

Core rule: Character streams are intended for text. Use byte streams for raw binary data such as images, audio, video and exact binary copies.

Contents

• Lesson 6 — BufferedReader

• Lesson 7 — BufferedWriter

• Lesson 8 — InputStreamReader

• Lesson 9 — OutputStreamWriter

• Lesson 10 — PrintWriter

• Lesson 11 — Character Encoding

• Lesson 12 — Complete Hierarchy

• Lesson 13 — Class Comparisons

• Lesson 14 — Practical Programs

• Lesson 15 — Mistakes and Interview Questions

• Final Revision Sheet

Lesson 6 — BufferedReader

Definition and purpose

BufferedReader is a character-input stream class that reads text from another Reader efficiently by buffering characters. It also provides readLine() to read a complete line. Package: java.io. Parent: Reader. It is a concrete class.

Inheritance

```
Object
  |
Reader
  |
BufferedReader
```

BufferedReader wraps another Reader. For file input, FileReader decodes bytes into characters and BufferedReader adds buffering and line-reading support.

Constructors

Constructor	Description
BufferedReader(Reader in)	Creates a reader with a default-sized buffer.
BufferedReader(Reader in, int size)	Creates a reader with the specified buffer size.

Important methods

Method	Description
read()	Reads one character; returns -1 at EOF.
read(char[])	Reads characters into an array.
read(char[], off, len)	Reads into a specified array range.
readLine()	Returns the next line as a String, or null at EOF.
ready()	Checks whether reading can proceed without blocking.

Method	Description
skip(long n)	Skips characters.
mark(int limit), reset()	Marks and returns to a position if supported.
markSupported()	Checks mark/reset support.
lines()	Returns a Stream of lines.
close()	Closes the reader.

Remember: readLine() excludes the line separator. It returns null at EOF, not an empty String.

Program — Read a file line by line

Example student.txt:

```
Jaideep
Java Programming
Computer Science
```

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class BufferedReaderExample {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(
                new FileReader("student.txt", StandardCharsets.UTF_8))) {
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Output is the three lines shown in the input file. FileReader decodes using UTF-8; BufferedReader wraps it; the loop ends when readLine() returns null.

Program — Count lines

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class LineCount {
    public static void main(String[] args) {
        int count = 0;
        try (BufferedReader br = new BufferedReader(
                new FileReader("student.txt", StandardCharsets.UTF_8))) {
            while (br.readLine() != null) count++;
            System.out.println("Total Lines: " + count);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

For the sample file, output: Total Lines: 3.

Program — Count words

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class WordCount {
    public static void main(String[] args) {
        int words = 0;
        try (BufferedReader br = new BufferedReader(
                new FileReader("student.txt", StandardCharsets.UTF_8))) {
            String line;
            while ((line = br.readLine()) != null) {
                line = line.trim();
                if (!line.isEmpty()) words += line.split("\\s+").length;
            }
            System.out.println("Total Words: " + words);
        } catch (IOException e) {
            e.printStackTrace();
```

```
        }
    }
}
```

Output for the sample: Total Words: 6. trim() removes outer whitespace; split("\\s+") splits on one or more whitespace characters. This is a simple whitespace-based count, not linguistic tokenization.

Program — Read keyboard input

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class KeyboardInput {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(
            new InputStreamReader(System.in, StandardCharsets.UTF_8));
        System.out.print("Enter your name: ");
        String name = br.readLine();
        System.out.println("Welcome " + name);
    }
}
```

System.in is a byte InputStream. InputStreamReader decodes bytes into characters. BufferedReader buffers characters and provides readLine(). Avoid closing this wrapper if the rest of the application still needs System.in.

Lesson 7 — BufferedWriter

Definition and inheritance

BufferedWriter is a character-output stream that efficiently writes text to another Writer by temporarily storing characters in a buffer. Package: java.io. Parent: Writer.

```
Object
  |
Writer
  |
BufferedWriter
```

Constructors

Constructor	Description
BufferedWriter(Writer out)	Uses a default-sized buffer.
BufferedWriter(Writer out, int size)	Uses the specified buffer size.

Important methods

Method	Description
write(int c)	Writes one character.
write(char[])	Writes a character array.
write(String)	Writes a String.
write(String, off, len)	Writes part of a String.
append(...)	Appends character data.
newLine()	Writes the platform-specific line separator.
flush()	Forces buffered output to the underlying writer.
close()	Flushes and closes the writer.

Program — Write text

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class BufferedWriterExample {
    public static void main(String[] args) {
        try (BufferedWriter bw = new BufferedWriter(
                new FileWriter("output.txt", StandardCharsets.UTF_8))) {
```

```
            bw.write("Java Programming");
            bw.newLine();
            bw.write("Python Programming");
            bw.newLine();
            bw.write("C Programming");
            System.out.println("Data written successfully.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

The file contains three lines. newLine() writes the platform-specific line separator.

Program — Append text

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class AppendExample {
    public static void main(String[] args) {
        try (BufferedWriter bw = new BufferedWriter(
                new FileWriter("output.txt", StandardCharsets.UTF_8, true))) {
            bw.newLine();
            bw.write("Java Full Stack Development");
            System.out.println("Data appended successfully.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

The final true enables append mode. Without append mode, opening an existing file for writing normally truncates its contents.

Program — Copy a text file line by line

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class TextFileCopy {
    public static void main(String[] args) {
        try (
            BufferedReader br = new BufferedReader(
                new FileReader("source.txt", StandardCharsets.UTF_8));
            BufferedWriter bw = new BufferedWriter(
                new FileWriter("destination.txt", StandardCharsets.UTF_8))
        ) {
            String line;
            while ((line = br.readLine()) != null) {
                bw.write(line);
                bw.newLine();
            }
            System.out.println("File copied successfully.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Limitation: Line-based copying can normalize line endings and add a final line separator. For exact byte-for-byte copying, use byte streams.

BufferedReader	BufferedWriter
Reads text; extends Reader.	Writes text; extends Writer.
Input operation.	Output operation.
readLine().	newLine().
Wraps another Reader.	Wraps another Writer.

Lesson 8 — InputStreamReader

Definition: InputStreamReader is a bridge from byte streams to character streams. It reads bytes from an InputStream and decodes them into characters using a charset. Package: java.io. Parent: Reader.

```
Object
  |
Reader
  |
InputStreamReader
  |
FileReader
```

FileReader is a specialized InputStreamReader for files. System.in is a byte stream, so InputStreamReader is useful for turning console input into characters.

Constructors

Constructor	Description
InputStreamReader(InputStream in)	Uses the default charset.
InputStreamReader(InputStream in, Charset cs)	Uses the specified Charset.
InputStreamReader(InputStream in, String charsetName)	Uses the named charset.

Methods

Method	Description
read(), read(char[])	Reads characters.
ready()	Checks whether the stream is ready to be read.
getEncoding()	Returns the encoding name, or null if closed.
close()	Closes the reader and underlying stream.

Program — Read one character

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class InputStreamReaderExample {
    public static void main(String[] args) throws IOException {
        InputStreamReader isr = new InputStreamReader(
            System.in, StandardCharsets.UTF_8);
        System.out.print("Enter a character: ");
        int data = isr.read();
        System.out.println("Character: " + (char) data);
    }
}
```

For ordinary English letters this reads as expected. Technically read() returns one UTF-16 code unit; supplementary Unicode characters can require two reads.

Program — Read a complete line

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class KeyboardInputExample {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(
            new InputStreamReader(System.in, StandardCharsets.UTF_8));

        System.out.print("Enter your name: ");
        String name = br.readLine();
        System.out.print("Enter your college: ");
        String college = br.readLine();

        System.out.println("Name: " + name);
        System.out.println("College: " + college);
    }
}
```

The common console-input chain is System.in → InputStreamReader → BufferedReader.

Lesson 9 — OutputStreamWriter

Definition: OutputStreamWriter is a bridge from character streams to byte streams. It encodes characters into bytes using a charset and writes those bytes to an OutputStream. Package: java.io. Parent: Writer.

```
Object
  |
Writer
  |
OutputStreamWriter
  |
FileWriter
```

Constructors

Constructor	Description
OutputStreamWriter(OutputStream out)	Uses the default charset.
OutputStreamWriter(OutputStream out, Charset cs)	Uses the specified Charset.
OutputStreamWriter(OutputStream out, String charsetName)	Uses the named charset.

Methods

Method	Description
write(int), write(char[]), write(String)	Writes character data.
append(...)	Appends character data.
flush()	Forces pending encoded output to the underlying stream.
close()	Closes writer and underlying stream.
getEncoding()	Returns the encoding name, or null if closed.

Program — Write text

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class OutputStreamWriterExample {
    public static void main(String[] args) {
        try (OutputStreamWriter osw = new OutputStreamWriter(
                new FileOutputStream("output.txt"), StandardCharsets.UTF_8)) {
            osw.write("Java File Handling");
            osw.write(System.lineSeparator());
            osw.write("Character Streams");
            System.out.println("Data written successfully.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

OutputStreamWriter encodes characters as UTF-8 bytes; FileOutputStream writes those bytes.

Combine buffering and encoding

```
BufferedWriter bw = new BufferedWriter(
    new OutputStreamWriter(
        new FileOutputStream("data.txt"),
        StandardCharsets.UTF_8
    )
);
```

BufferedWriter provides buffering; OutputStreamWriter performs encoding; FileOutputStream writes bytes.

Lesson 10 — PrintWriter

Definition: PrintWriter is a character-output stream class providing convenient methods to print formatted representations of objects and primitive values. Package: java.io. Parent: Writer.

```
Object
  |
Writer
  |
```

`PrintWriter`

Constructors

Constructor	Description
PrintWriter(String fileName)	Writes to a named file.
PrintWriter(File file)	Writes to a file.
PrintWriter(Writer out)	Wraps another Writer.
PrintWriter(Writer out, boolean autoFlush)	Writer wrapper with optional auto-flush.
PrintWriter(OutputStream out)	Wraps an OutputStream using default charset.
PrintWriter(OutputStream out, boolean autoFlush)	OutputStream wrapper with auto-flush option.
PrintWriter(OutputStream out, boolean autoFlush, Charset cs)	Uses a specified charset.

Methods

Method	Description
print()	Prints without automatically adding a line separator.
println()	Prints followed by a line separator.
printf(), format()	Print formatted output.
append()	Appends character data.
write()	Writes character data.
flush(), close()	Flushes or closes the writer.
checkError()	Checks whether an output error has occurred.

Program — Write student data

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class PrintWriterExample {
    public static void main(String[] args) {
        try (PrintWriter pw = new PrintWriter(
                "student.txt", StandardCharsets.UTF_8)) {
            pw.println("Name: Jaideep");
            pw.println("Course: B.Tech CSE");
            pw.println("College: Sri Vasavi Engineering College");
            System.out.println("Data written successfully.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Program — printf formatting

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class PrintFormatting {
    public static void main(String[] args) {
        try (PrintWriter pw = new PrintWriter(
                "marks.txt", StandardCharsets.UTF_8)) {
            String name = "Jaideep";
            int marks = 95;
            double percentage = 95.75;

            pw.printf("Name: %s%n", name);
            pw.printf("Marks: %d%n", marks);
            pw.printf("Percentage: %.2f%%%n", percentage);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

File output: Name: Jaideep; Marks: 95; Percentage: 95.75%.

Specifier	Meaning
%s	String
%d	Integer
%f	Floating-point value
%.2f	Two decimal places
%c	Character
%n	Platform-specific line separator
%%	Literal percent sign

PrintWriter vs BufferedWriter

PrintWriter	BufferedWriter
Convenient print and formatted output.	Buffered character writing.
print(), println(), printf().	write() and newLine().
Conveniently prints primitive values and objects.	Primarily writes character data.
Suppresses write IOExceptions; use checkError().	Writing methods can throw IOException.

Important: PrintWriter suppresses IOExceptions from writing operations and records an error state. Use checkError() when you need to detect output errors. File-opening constructors can still throw FileNotFoundException.

Lesson 11 — Character Encoding

Definition

Character encoding represents characters as bytes for storage or transmission. Decoding converts encoded bytes back into characters.

Operation	Conversion
Encoding	Characters → Bytes
Decoding	Bytes → Characters

Unicode

Unicode is a universal character standard assigning code points to characters across writing systems. Examples: A (U+0041), B (U+0042), ■ (U+20B9). Unicode defines characters and code points; UTF-8 and UTF-16 define encoding schemes.

UTF-8

UTF-8 is a variable-length Unicode encoding using one to four bytes per Unicode code point. ASCII characters use one byte. Many Indian-language characters use three bytes; some emoji use four bytes.

UTF-16

UTF-16 represents Unicode characters using one or two 16-bit code units. Many common characters use one code unit (two bytes); supplementary characters use a surrogate pair (four bytes).

Feature	UTF-8	UTF-16
Basic unit	8 bits	16 bits
Length	1–4 bytes/code point	2 or 4 bytes/code point
ASCII character	1 byte	2 bytes

Feature	UTF-8	UTF-16
ASCII byte compatibility	Yes	No
Common use	Web and general text	Some text systems and APIs

Java char detail: char is a 16-bit UTF-16 code unit, not always a complete Unicode code point. Supplementary characters require a surrogate pair.

Charset in Java

java.nio.charset.Charset represents a named character encoding. StandardCharsets provides constants such as UTF_8, UTF_16, UTF_16BE, UTF_16LE, US_ASCII and ISO_8859_1.

```
import java.nio.charset.StandardCharsets;

public class CharsetExample {
    public static void main(String[] args) {
        String message = "Hello Java";
        byte[] data = message.getBytes(StandardCharsets.UTF_8);
        String result = new String(data, StandardCharsets.UTF_8);
        System.out.println(result);
    }
}
```

Output: Hello Java.

Why specify UTF-8 explicitly?

```
// Uses the environment's default charset:
FileWriter fw1 = new FileWriter("data.txt");

// Explicit charset:
FileWriter fw2 = new FileWriter(
    "data.txt", StandardCharsets.UTF_8);

// Read with the matching charset:
FileReader fr = new FileReader(
    "data.txt", StandardCharsets.UTF_8);
```

The charset used to read should match the charset used to write. A mismatch can corrupt text. Avoid relying on the platform default when a file format requires a particular encoding.

Lesson 12 — Complete Character Stream Hierarchy

Input hierarchy

```
Object
  |
Reader (abstract)
  |-------------------------|
  |                         |
InputStreamReader        BufferedReader
  |
FileReader
```

FileReader extends InputStreamReader, which extends Reader. BufferedReader also extends Reader and is commonly wrapped around FileReader or InputStreamReader.

Output hierarchy

```
Object
  |
Writer (abstract)
  |-------------------------------|
  |               |               |
OutputStreamWriter BufferedWriter PrintWriter
  |
FileWriter
```

FileWriter extends OutputStreamWriter, which extends Writer. BufferedWriter and PrintWriter also extend Writer.

Inheritance vs wrapping

```
BufferedReader br = new BufferedReader(
    new InputStreamReader(System.in, StandardCharsets.UTF_8)
```

```
);
```

Here BufferedReader wraps InputStreamReader. They are not in a parent-child inheritance relationship. Inheritance describes class extension; wrapping is composition at runtime.

Lesson 13 — Class Comparisons

Class	Main purpose	Key methods
Reader	Abstract character input	read(), close()
Writer	Abstract character output	write(), flush()
FileReader	Read text files	read()
FileWriter	Write text files	write()
BufferedReader	Buffered character input	readLine()
BufferedWriter	Buffered character output	newLine()
InputStreamReader	Bytes → characters	read(), getEncoding()
OutputStreamWriter	Characters → bytes	write(), getEncoding()
PrintWriter	Convenient formatted output	print(), println(), printf()

FileReader vs BufferedReader vs InputStreamReader

Feature	FileReader	BufferedReader	InputStreamReader
Purpose	File character input	Buffered character input	Byte-to-character conversion
Reads from	File	Another Reader	InputStream
Charset	Can specify Charset	Uses wrapped Reader's decoding	Can specify Charset
readLine()	No	Yes	No

Output class comparison

Feature	FileWriter	BufferedWriter	OutputStreamWriter	PrintWriter
Purpose	File character output	Buffered character output	Character-to-byte conversion	Convenient text output
Destination	File	Another Writer	OutputStream	Writer/stream/file
Formatted output	No	No	No	Yes
Special method	write()	newLine()	write()	println(), printf()

Lesson 14 — Practical Programs

Program 1 — Copy a text file

Use BufferedReader and BufferedWriter to read each source line and write it to the destination. This line-based copy may normalize line endings; use byte streams when exact file preservation is required.

Program 2 — Count lines, words and characters

Read one line at a time. Increment line count for every non-null line. Use trim() and split("\\s+") for a basic whitespace word count. For character count, decide whether line separators should be included: readLine() removes them.

Program 3 — Search for a word

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class SearchWord {
    public static void main(String[] args) {
        String target = "Java";
        try (BufferedReader br = new BufferedReader(
```

```
                new FileReader("data.txt", StandardCharsets.UTF_8))) {
            String line;
            int lineNumber = 0;
            while ((line = br.readLine()) != null) {
                lineNumber++;
                if (line.contains(target)) {
                    System.out.println("Found at line: " + lineNumber);
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

For input 'Java is popular.', 'Python is easy.', 'Java supports OOP.', output is Found at line: 1 and Found at line: 3.

Program 4 — Store student details

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class StudentDetails {
    public static void main(String[] args) {
        try (PrintWriter pw = new PrintWriter(
                "student.txt", StandardCharsets.UTF_8)) {
            pw.println("Student ID: 101");
            pw.println("Name: Jaideep");
            pw.println("Department: CSE");
            pw.println("CGPA: 8.8");
            System.out.println("Student details saved.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Program 5 — Read student details

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class ReadStudent {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(
                new FileReader("student.txt", StandardCharsets.UTF_8))) {
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Lesson 15 — Common Mistakes and Interview Questions

Common mistakes

• Using FileReader for images or videos. Use byte streams for binary data.

• Forgetting to close streams. Prefer try-with-resources.

• Forgetting append mode. File-writing constructors normally truncate existing content unless append is enabled.

• Assuming readLine() returns an empty String at EOF. It returns null.

• Assuming BufferedReader is file-specific. It can wrap any suitable Reader.

• Ignoring character encoding. Specify UTF-8 explicitly when appropriate.

• Assuming PrintWriter throws every writing error as IOException. It suppresses writing exceptions; use checkError().

• Closing a wrapper around System.in and expecting to keep using System.in later.

Interview questions and answers

1. Difference between byte streams and character streams?

Byte streams handle raw bytes and are commonly used for binary data. Character streams handle text and perform encoding/decoding.

2. Purpose of BufferedReader?

It improves reading efficiency through buffering and provides readLine() to read complete lines.

3. Difference between read() and readLine()?

read() reads one character and returns an int (or -1 at EOF). readLine() returns a String or null at EOF.

4. Why is InputStreamReader a bridge class?

It converts byte input from an InputStream into character input for Reader-based APIs.

5. Why is OutputStreamWriter a bridge class?

It converts characters from a Writer into encoded bytes for an OutputStream.

6. BufferedWriter vs PrintWriter?

BufferedWriter provides buffered character writing. PrintWriter provides print(), println() and printf().

7. flush() vs close()?

flush() sends buffered output onward but leaves the stream open. close() flushes and closes it.

8. What is UTF-8?

A variable-length Unicode encoding using one to four bytes per Unicode code point.

9. What happens if FileWriter is used without a charset?

It uses the environment's default charset.

10. Can BufferedReader read keyboard input?

Yes. Wrap System.in in InputStreamReader, then wrap that in BufferedReader.

Final Revision Sheet

Input class	Remember this
Reader	Abstract character input class
FileReader	Reads text files
BufferedReader	Buffered reading; readLine()
InputStreamReader	Converts bytes to characters

Output class	Remember this
Writer	Abstract character output class
FileWriter	Writes text files
BufferedWriter	Buffered writing; newLine()
OutputStreamWriter	Converts characters to bytes
PrintWriter	Convenient and formatted output

Method	Class
readLine()	BufferedReader
newLine()	BufferedWriter

Method	Class
print(), println(), printf()	PrintWriter
getEncoding()	InputStreamReader / OutputStreamWriter
flush()	Writer and subclasses
close()	Reader / Writer and subclasses

Mastery checkpoint: You should be able to explain the Reader/Writer hierarchy, distinguish inheritance from wrapping, read lines, append text, copy a text file, read console input through InputStreamReader, write encoded text through OutputStreamWriter, and format output with PrintWriter.

Self-practice checklist

■ Read a text file using FileReader and BufferedReader.

■ Count lines and whitespace-separated words in a file.

■ Write three lines using BufferedWriter and newLine().

■ Append text without deleting existing contents.

■ Read a complete keyboard line using InputStreamReader and BufferedReader.

■ Write UTF-8 text using OutputStreamWriter and FileOutputStream.

■ Create a student report using PrintWriter and printf().

■ Explain why text files need a consistent charset and binary files should use byte streams.